



Universidad Nacional Autónoma De México

Facultad de Ingeniería

Reporte Técnico

Presenta:

Aquino Martínez Alexis Emmanuel

Cortes Suarez Alexis

Pérez Cruz Emilio Alejandro

Verona Lara Alan Jahir

Profesor:

Ing. Ariel Adara Mercado Martínez

Materia:

Estructura y Programación de
Computadores.

Fecha:

11 de junio de 2026

Introducción

Este proyecto implementa un ensamblador y un mini linker para un subconjunto de la arquitectura IA-32 (la de los procesadores Intel de 32 bits). La idea es simple: convertir un archivo de texto con instrucciones en ensamblador un archivo binario que una computadora pueda ejecutar.

Todo el código está escrito en C puro, sin usar herramientas automáticas como Flex, Bison o Yacc. Esto nos obligó a hacer todo manualmente, pero así aprendimos realmente cómo funcionan los compiladores y enlazadores de bajo nivel.

Lexer

Recibe el archivo .asm como texto puro y lo corta en pedazos pequeños llamados tokens. Un token es como una palabra en un idioma: puede ser una instrucción (MOV), un registro (EAX), un número (10), una coma (,), un corchete ([), etc.

El Lexer lee el archivo carácter por carácter usando la función fgetc(). Va acumulando letras hasta encontrar un separador (espacio, tabulador, salto de línea, o un símbolo especial). Cuando tiene un grupo completo, decide qué tipo de token es:

- **Token_Register:** Se usa si es una palabra y está en la lista de registros (EAX, EBX, ECX, EDX, ESI, EDI, EBP, ESP)
- **Token_Identifier:** Se usa si una palabra que no es registro (MOV, JMP)
- **Token_Number:** Solo si son dígitos.
- **Token_Colon:** con texto ":"
- **Token_Comma:** con texto ","
- **Token_Semicolon:** con texto ";"

Parser

El parser recibe la lista de tokens y la organiza en una estructura con significado. Verifica que la sintaxis sea correcta: por ejemplo, después de MOV debe venir un destino, luego una coma y luego un origen. También detecta el **modo de direccionamiento**: no es lo mismo MOV EAX, 10 (inmediato) que MOV EAX, [EBX+4] (memoria con desplazamiento). En nuestro proyecto, el parser y el backend están integrados en un solo módulo. Las funciones `parse_program` (primera pasada) y `pass2_program` (segunda pasada) analizan la sintaxis y también construyen la tabla de símbolos. No hay un parser separado que genere una estructura `Instruction` completa; esa estructura la arma el `main.c` del pipeline final usando el lexer, el parser extendido y la tabla de símbolos.

Estructura interna: El parser original se configuraba de la siguiente forma.

- **opcode:** El nombre de la instrucción ("MOV")
- **has_memory:** Si el operando usa corchetes (memoria)
- **immediate:** Si hay un número inmediato.
- **has_displacement:** 1 si hay un número sumado o restado dentro de los corchetes (ej. [EBX+8]).
- **base_reg:** nombre del registro base (ej. "EBX", "EBP").
- **index_reg:** nombre del registro índice (ej. "ECX").
- **scale:** escala del SIB (1, 2, 4, 8). Si no hay índice, scale = 0

Backend

Hace dos pasadas completas sobre el archivo. La primera pasada construye una tabla de símbolos (un directorio de todas las etiquetas y sus direcciones). La segunda pasada genera el código máquina (los bytes) y resuelve las referencias adelantadas (saltos a etiquetas que aún no se habían definido). Este mecanismo se llama fixup.

Tabla de símbolo: Es un arreglo simple con espacios para símbolos, es definida por `symtab.h`

- **name:** El nombre de la etiqueta.
- **address:** La dirección donde se encuentra.
- **defined:** 1 si ya se definió, 0 si solo se usó.
- **is_extern:** 1 si fue declarado con `EXTERN`, 0 si es una etiqueta local o global.
- **init_symtab():** Limpia la tabla.
- **add_symbol(name, address, defined):** Agrega un símbolo o actualiza uno existente (resuelve referencias adelantadas). Si intentas definir dos veces el mismo símbolo, da error.
- **add_extern_symbol(name):** agrega un símbolo externo (sin dirección, `defined=0`, `is_extern=1`).
- **get_symbol(name):** Busca y devuelve el símbolo.
- **print_symtab():** muestra la tabla para depuración.

Primera pasada: Recorre el archivo usando el lexer, mantiene un “Location Counter” que empieza en 0 y va incrementandose según el tamaño de cada instrucción. Cuando encuentra un identificador seguido de dos puntos (:), eso es una etiqueta. La agrega a la tabla de símbolos con la dirección actual del LC y lo define. Cuando encuentra una instrucción (`MOV`, `ADD`, `JMP`, etc.), analiza sus operandos y aumenta el LC

según el tamaño de esa instrucción. Al final de la primera pasada, imprime la tabla de símbolos.

Segunda pasada: Rebobina el archivo y vuelve a leerlo. Abre un archivo “**output.hex**” donde escribirá el código máquina en formato texto (cada línea muestra la dirección y los bytes en hexadecimal). Recorre las instrucciones de nuevo, pero ahora ya conoce las direcciones de todas las etiquetas. Para cada instrucción, calcula los bytes reales (usando el encoder si está disponible, o emitiendo opcodes simulados como 8B para MOV) y los escribe en output.hex. El resultado es un archivo output.hex que muestra el código máquina generado.

Encoder IA-32

El encoder recibe una instrucción ya completamente estructurada (con operandos clasificados) y la convierte en los bytes reales que el procesador IA-32 entiende. No es trivial, porque cada instrucción tiene diferentes formas: por ejemplo, MOV EAX, 10 usa un opcode distinto a MOV [EBX+4], EAX. Además hay que construir bytes especiales llamados ModRM y SIB para direccionamientos complejos como [EBX+ECX*4+8].

Estructura clave: Es definida por "ia32_types.h" y tiene esta parte la siguiente estructura.

- **Mnemonic:** Un enumerado con los nombres de instrucción (MN_MOV, MN_JMP, etc.).
- **dst y src:** Dos operandos, cada uno con su tipo (OP_REG, OP_IMM, OP_MEM_SIB, etc.), registros, índices, escala, desplazamiento, inmediato, y una bandera needs_reloc que indica si el símbolo es externo.
- **address:** la dirección donde se colocará esta instrucción.

Primitivas de bajo nivel: Es definida por "encoder.c"

- **encoder_build_modrm(mod, reg, rm):** Ensambla un byte ModRM.
- **encoder_build_sib(scale, index, base):** ensambla el byte SIB.
- **encoder_write_le32(buf, value):** escribe un número de 32 bits en formato little-endian (el byte más pequeño primero).

Codificación de operandos de memoria: Es definida por "encoder_encode_mem_operand". Toma un operando de memoria (como [EBX+ECX*4+8]) y emite los bytes necesarios. Decide si el desplazamiento cabe en 8 bits con signo o necesita 32 bits. Para casos especiales [ESP] requiere SIB aunque no haya índice; [EBP] sin desplazamiento se fuerza a [EBP+0] para evitar confusiones con direcciones absolutas.

Encoders por familia de instrucción: Cada encoder tiene una función separa para cada familia.

- **encoder_encode_mov:** maneja 4 variantes de MOV
- **encoder_encode_alu:** para ADD, SUB, AND, OR, XOR, CMP.
Dependiendo de los operandos elige entre tres formas (la que usa registro inmediato, la de registro a registro, etc.).
- **encoder_encode_unary:** para PUSH, POP, INC, DEC, NOT, NEG, MUL, DIV.
Usa formas cortas de registro cuando puede (por ejemplo 0x50+rd para PUSH EAX) o el opcode 0xF7 con un /digit en el ModR
- **ncoder_encode_jump y encoder_encode_call:** Generan saltos relativos de 32 bits. Para JMP usan 0xE9, para saltos condicionales 0x0F 0x8X, para CALL usan 0xE8.

Formato objeto y linker

Esta sección se divide en 2 partes definir cómo se guarda un archivo objeto (.o) y luego escribir el linker que une varios .o en un solo ejecutable.

Archivo objeto: Cuando el encoder termina de procesar un archivo .asm, el resultado no es todavía un ejecutable. Es un **archivo objeto**, una caja que contiene cuatro compartimentos.

- **El encabezado (ObjHeader):** Son los metadatos de la caja. Tiene un número mágico (0x4F424A21, que en ASCII se lee "OBJ!") para identificar que el archivo es válido. También guarda cuántas secciones, cuántos símbolos y cuántas relocalaciones hay.
- **Las secciones (Section):** Hay tres tipos importantes
 - **.text:** donde vive el código máquina (los bytes que genera el encoder).
 - **.data:** donde viven las variables inicializadas (como las que se definen con DB o DW).
 - **.bss:** donde viven las variables no inicializadas (como RESB 10), que solo reservan espacio pero no ocupan bytes en el archivo

Cada sección tiene un nombre, un arreglo de bytes (de hasta 4096) y un tamaño.

- **La tabla de símbolos (Symbol):** Es un directorio. Cada entrada tiene el nombre del símbolo, su offset dentro de su sección, y su tipo:
 - **LOCAL:** solo visible dentro de este archivo objeto.
 - **GLOBAL:** visible para otros archivos (cuando usas la directiva GLOBAL en el .asm).
 - **EXTERN:** definido en otro archivo (directiva EXTERN).

- **La tabla de relocaciones (Relocation):** Es una lista de "parches pendientes". Cada entrada dice: "en el offset X de la sección .text hay un placeholder de 4 bytes, que corresponde al símbolo Y, y cuando el linker sepa dónde queda Y, debe escribir ahí la dirección". Hay dos tipos de relocación:
 - **REL32:** dirección relativa de 32 bits (para CALL y JMP a funciones externas).
 - **ABS32:** dirección absoluta de 32 bits (para referencias a variables en memoria).

Funciones que implementé para manejar objetos: Están definidos en `object.c`.

- **obj_create():** crea un objeto vacío en memoria.
- **obj_add_section():** agrega una nueva sección (.text, .data, .bss).
- **obj_write_bytes():** escribe bytes en una sección (así el encoder va depositando el código).
- **obj_add_symbol():** agrega un símbolo con su tipo.
- **obj_add_reloc():** agrega una relocación.
- **obj_write_file():** guarda todo el objeto en un archivo binario (.o). Escribe primero el encabezado, luego las secciones, luego los símbolos, luego las relocaciones.
- **obj_read_file():** lee un archivo .o y lo reconstruye en memoria (lo usa el linker).
- **obj_print():** muestra en pantalla el contenido del objeto para depuración.

Linker: El linker recibe dos o más archivos objeto y los une en un solo binario ejecutable. Trabaja en **cuatro pasos** bien definidos. Todo el estado del linker está en una estructura LinkerCtx que guarda la lista de objetos cargados, la tabla global de símbolos, los offsets de las secciones fusionadas y el buffer de salida.

- **Paso 1:** Toma todos los .text de todos los objetos y los pone uno tras otro en el buffer de salida. Luego hace lo mismo con todos los .data, y finalmente con .bss (pero .bss solo reserva espacio, no escribe bytes).
- **Paso 2:** Construir la tabla global de símbolos (linker_build_symbol_table) Recorre todos los símbolos de todos los objetos. Solo le interesan los GLOBAL y los EXTERN.
- **Paso 3:** Aplicar relocalaciones (linker_apply_relocations) Para cada relocalación de cada objeto. Localiza la posición exacta en el buffer de salida donde está el placeholder. Para eso usa section_offsets para saber dónde empieza la sección .text de ese objeto, y le suma el offset de la relocalación. Busca el símbolo destino en la tabla global de símbolos. Si es ABS32, simplemente escribe la dirección del símbolo (4 bytes) Y sobrescribe los bytes en el buffer de salida.
- **Paso 4:** Escribir el binario final (linker_write_binary). Guarda el buffer de salida (que ya contiene todas las secciones fusionadas y las relocalaciones aplicadas) en un archivo, por ejemplo programa.bin. Ese archivo es el ejecutable final (en formato binario plano).